JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Bidi

java.lang.Object
    java.text.Bidi

```
public final class Bidi
extends Object
```

This class implements the Unicode Bidirectional Algorithm.

A Bidi object provides information on the bidirectional reordering of the text used to create it. This is required, for example, to properly display Arabic or Hebrew text. These languages are inherently mixed directional, as they order numbers from left-to-right while ordering most other text from right-to-left.

Once created, a Bidi object can be queried to see if the text it represents is all left-to-right or all right-to-left. Such objects are very lightweight and this text is relatively easy to process.

If there are multiple runs of text, information about the runs can be accessed by indexing to get the start, limit, and level of a run. The level represents both the direction and the 'nesting level' of a directional run. Odd levels are right-to-left, while even levels are left-to-right. So for example level 0 represents left-to-right text, while level 1 represents right-to-left text, and level 2 represents left-to-right text embedded in a right-to-left run.

Since:

1.4

Field Summary

Fields

Modifier and Type	Field	Description
`static final int`	`DIRECTION_DEFAULT_LEFT_T`	Constant indicating that the base direction depends on the first strong directional character in the text according to the Unicode Bidirectional Algorithm.
`static final int`	`DIRECTION_DEFAULT_RIGHT_`	Constant indicating that the base direction depends on the first strong directional character in the text

	according to the Unicode Bidirectional Algorithm.
`static final int` **DIRECTION_LEFT_TO_RIGHT**	Constant indicating base direction is left-to-right.
`static final int` **DIRECTION_RIGHT_TO_LEFT**	Constant indicating base direction is right-to-left.

Constructor Summary

Constructors

Constructor	Description
Bidi(char[] text, int textStart, byte[] embeddings, int embStart, int paragraphLength, int flags)	Create Bidi from the given text, embedding, and direction information.
Bidi(String paragraph, int flags)	Create Bidi from the given paragraph of text and base direction.
Bidi(AttributedCharacterIterator parag	Create Bidi from the given paragraph of text.

Method Summary

All Methods | **Static Methods**

Modifier and Type	Method	Description
boolean	**baseIsLeftToRight**()	Return true if the base direction is left-to-right.
Bidi	**createLineBidi**(int lineStart, int lineLimit)	Create a Bidi object representing the bidi information on a line of text within the paragraph represented by the current Bidi.

int	**getBaseLevel**()	Return the base level (0 if left-to-right, 1 if right-to-left).
int	**getLength**()	Return the length of text in the line.
int	**getLevelAt**(int offset)	Return the resolved level of the character at offset.
int	**getRunCount**()	Return the number of level runs.
int	**getRunLevel**(int run)	Return the level of the nth logical run in this line.
int	**getRunLimit**(int run)	Return the index of the character past the end of the nth logical run in this line, as an offset from the start of the line.
int	**getRunStart**(int run)	Return the index of the character at the start of the nth logical run in this line, as an offset from the start of the line.
boolean	**isLeftToRight**()	Return true if the line is all left-to-right text and the base direction is left-to-right.
boolean	**isMixed**()	Return true if the line is not left-to-right or right-to-left.
boolean	**isRightToLeft**()	Return true if the line is all right-to-left text, and the base direction is right-to-left.
static void	**reorderVisually**(byte[] levels, int levelStart,	Reorder the objects in the array into visual

	Object[] objects, int objectStart, int count)	order based on their levels.
static boolean	**requiresBidi** (char[] text, int start, int limit)	Return true if the specified text requires bidi analysis.
String	**toString**()	Display the bidi internal state, used in debugging.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Field Details

DIRECTION_LEFT_TO_RIGHT

```
public static final int DIRECTION_LEFT_TO_RIGHT
```

Constant indicating base direction is left-to-right.

See Also:

Constant Field Values

DIRECTION_RIGHT_TO_LEFT

```
public static final int DIRECTION_RIGHT_TO_LEFT
```

Constant indicating base direction is right-to-left.

See Also:

Constant Field Values

DIRECTION_DEFAULT_LEFT_TO_RIGHT

```
public static final int DIRECTION_DEFAULT_LEFT_TO_RIGHT
```

Constant indicating that the base direction depends on the first strong directional character in the text according to the Unicode Bidirectional Algorithm. If no strong directional character is present, the base direction is left-to-right.

See Also:

Constant Field Values

DIRECTION_DEFAULT_RIGHT_TO_LEFT

```
public static final int DIRECTION_DEFAULT_RIGHT_TO_LEFT
```

Constant indicating that the base direction depends on the first strong directional character in the text according to the Unicode Bidirectional Algorithm. If no strong directional character is present, the base direction is right-to-left.

See Also:

Constant Field Values

Constructor Details

Bidi

```
public Bidi(String paragraph,
            int flags)
```

Create Bidi from the given paragraph of text and base direction.

Parameters:

`paragraph` - a paragraph of text

`flags` - a collection of flags that control the algorithm. The algorithm understands the flags DIRECTION_LEFT_TO_RIGHT, DIRECTION_RIGHT_TO_LEFT, DIRECTION_DEFAULT_LEFT_TO_RIGHT, and DIRECTION_DEFAULT_RIGHT_TO_LEFT. Other values are reserved.

Bidi

```
public Bidi(AttributedCharacterIterator paragraph)
```

Create Bidi from the given paragraph of text.

The RUN_DIRECTION attribute in the text, if present, determines the base direction (left-to-right or right-to-left). If not present, the base direction is computed using the Unicode Bidirectional Algorithm, defaulting to left-to-right if there are no strong directional characters in the text. This attribute, if present, must be applied to all the text in the paragraph.

The BIDI_EMBEDDING attribute in the text, if present, represents embedding level information. Negative values from -1 to -62 indicate overrides at the absolute value of the level. Positive values from 1 to 62 indicate embeddings.

Where values are zero or not defined, the base embedding level as determined by the base direction is assumed.

The NUMERIC_SHAPING attribute in the text, if present, converts European digits to other decimal digits before running the bidi algorithm. This attribute, if present, must be applied to all the text in the paragraph.

Parameters:

`paragraph` - a paragraph of text with optional character and paragraph attribute information

See Also:

`TextAttribute.BIDI_EMBEDDING`, `TextAttribute.NUMERIC_SHAPING`, `TextAttribute.RUN_DIRECTION`

Bidi

```
public Bidi(char[] text,
            int textStart,
            byte[] embeddings,
            int embStart,
            int paragraphLength,
            int flags)
```

Create Bidi from the given text, embedding, and direction information. The embeddings array may be null. If present, the values represent embedding level information. Negative values from -1 to -61 indicate overrides at the absolute value of the level. Positive values from 1 to 61 indicate embeddings. Where values are zero, the base embedding level as determined by the base direction is assumed.

Parameters:

`text` - an array containing the paragraph of text to process.

`textStart` - the index into the text array of the start of the paragraph.

`embeddings` - an array containing embedding values for each character in the paragraph. This can be null, in which case it is assumed that there is no external embedding information.

`embStart` - the index into the embedding array of the start of the paragraph.

`paragraphLength` - the length of the paragraph in the text and embeddings arrays.

`flags` - a collection of flags that control the algorithm. The algorithm understands the flags DIRECTION_LEFT_TO_RIGHT, DIRECTION_RIGHT_TO_LEFT, DIRECTION_DEFAULT_LEFT_TO_RIGHT, and DIRECTION_DEFAULT_RIGHT_TO_LEFT. Other values are reserved.

Method Details

createLineBidi

```
public Bidi createLineBidi(int lineStart,
                           int lineLimit)
```

Create a Bidi object representing the bidi information on a line of text within the paragraph represented by the current Bidi. This call is not required if the entire paragraph fits on one line.

Parameters:

`lineStart` - the offset from the start of the paragraph to the start of the line.

`lineLimit` - the offset from the start of the paragraph to the limit of the line.

Returns:

a `Bidi` object

isMixed

```
public boolean isMixed()
```

Return true if the line is not left-to-right or right-to-left. This means it either has mixed runs of left-to-right and right-to-left text, or the base direction differs from the direction of the only run of text.

Returns:

true if the line is not left-to-right or right-to-left.

isLeftToRight

```
public boolean isLeftToRight()
```

Return true if the line is all left-to-right text and the base direction is left-to-right.

Returns:

true if the line is all left-to-right text and the base direction is left-to-right

isRightToLeft

```
public boolean isRightToLeft()
```

Return true if the line is all right-to-left text, and the base direction is right-to-left.

Returns:

true if the line is all right-to-left text, and the base direction is right-to-left

getLength

```
public int getLength()
```

Return the length of text in the line.

Returns:

the length of text in the line

baseIsLeftToRight

```
public boolean baseIsLeftToRight()
```

Return true if the base direction is left-to-right.

Returns:

true if the base direction is left-to-right

getBaseLevel

```
public int getBaseLevel()
```

Return the base level (0 if left-to-right, 1 if right-to-left).

Returns:

the base level

getLevelAt

```
public int getLevelAt(int offset)
```

Return the resolved level of the character at offset. If offset is < 0 or ≥ the length of the line, return the base direction level.

Parameters:

`offset` - the index of the character for which to return the level

Returns:

the resolved level of the character at offset

getRunCount

```
public int getRunCount()
```

Return the number of level runs.

Returns:

the number of level runs

getRunLevel

```
public int getRunLevel(int run)
```

Return the level of the nth logical run in this line.

Parameters:

`run` - the index of the run, between 0 and `getRunCount()`

Returns:

the level of the run

getRunStart

```
public int getRunStart(int run)
```

Return the index of the character at the start of the nth logical run in this line, as an offset from the start of the line.

Parameters:

`run` - the index of the run, between 0 and `getRunCount()`

Returns:

the start of the run

getRunLimit

```
public int getRunLimit(int run)
```

Return the index of the character past the end of the nth logical run in this line, as an offset from the start of the line. For example, this will return the length of the line for the last run on the line.

Parameters:

`run` - the index of the run, between 0 and `getRunCount()`

Returns:

limit the limit of the run

requiresBidi

```
public static boolean requiresBidi(char[] text,
                                   int start,
                                   int limit)
```

Return true if the specified text requires bidi analysis. If this returns false, the text will display left-to-right. Clients can then avoid constructing a Bidi object. Text in the Arabic Presentation Forms area of Unicode is presumed to already be shaped and ordered for display, and so will not cause this function to return true.

Parameters:

`text` - the text containing the characters to test

`start` - the start of the range of characters to test

`limit` - the limit of the range of characters to test

Returns:

true if the range of characters requires bidi analysis

reorderVisually

```
public static void reorderVisually(byte[] levels,
                                   int levelStart,
                                   Object[] objects,
                                   int objectStart,
                                   int count)
```

Reorder the objects in the array into visual order based on their levels. This is a utility function to use when you have a collection of objects representing runs of text in logical order, each run containing text at a single level. The elements at `index` from `objectStart` up to `objectStart + count` in the objects array will be reordered into visual order assuming each run of text has the level indicated by the corresponding element in the levels array (at `index - objectStart + levelStart`).

Parameters:

`levels` - an array representing the bidi level of each object

`levelStart` - the start position in the levels array

`objects` - the array of objects to be reordered into visual order

`objectStart` - the start position in the objects array

`count` - the number of objects to reorder

toString

```
public String toString()
```

Display the bidi internal state, used in debugging.

Overrides:

`toString` in class `Object`

Returns:

a string representation of the object

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which

contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.

Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.

Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.

All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.